

# PHSDSC717P – Mathematical Methods IV Lab

Saurish Chakrabarty

Acharya Prafulla Chandra College

## Part I

# Assignments

## 1 Complex Roots

- (a) Using the two-dimensional Newton-Raphson method, find the cube roots of a complex number by forming two simultaneous equations from the real and imaginary parts.
- (b) Calculate the roots by de Moivre's formula and compare the results.
- (c) Find complex roots of an arbitrary algebraic equation.

---

HINTS AND RELEVANT PYTHON FUNCTIONS:

- For obtaining the modulus and argument of a complex number, you may use `cmath.polar`.
- 

## 2 Improper Integrals

Consider the integrals,  $I_c(R) = \oint_C f(z)dz$  and  $I_r(R) = \int_{-R}^R f(x)dx$ ,  $C$  being a closed contour going from  $-R$  to  $R$  along the real axis and coming back along a semi-circular contour of radius  $R$  centered at the origin and on the upper half of the complex plane. For each of the functions, (i)  $f(z) = \frac{1}{z^2 + 1}$  (ii)  $f(z) = \frac{e^{3iz}}{z^4 + 81}$  (iii)  $f(z) = \frac{e^{5iz}}{z^6 + 64}$ ,

(a) evaluate  $\int_{-\infty}^{\infty} f(x)dx \equiv \lim_{R \rightarrow \infty} I_r(R)$ ,

(b) calculate the contribution from the semi-circular arc,  $I_{sc}(R)$ , as  $R \rightarrow \infty$ , and,

(c) calculate  $\lim_{R \rightarrow \infty} I_c(R)$ .

---

HINTS AND RELEVANT PYTHON FUNCTIONS:

- Use Simpson's  $\frac{1}{3}$  rule for the integrals by writing your own code.
- 

## 3 Gram-Schmidt Orthogonalization

- (a) Write a function which takes a pair of  $n$ -component vectors as input and returns a pair of orthogonal vectors using Gram-Schmidt orthogonalization. Write a function to check the orthogonality of the vectors. (Your function should work for vectors with real or complex elements.)
  - (b) Write a function which takes any number of  $n$ -component vectors as input and returns the same number of orthogonal vectors using Gram-Schmidt orthogonalization. Write a function to check the orthogonality of the vectors by forming a suitable orthogonal/unitary matrix. Test your functions by reading vectors from rows of an input file and writing the output to another file.
-

## 4 Solution of Linear Systems of Equations using Gauss elimination and Gauss-Jordan methods

- (a) Write a function to calculate the determinant of a matrix using the Gauss elimination method.
- (b) Write a function to find the solution of linear system of equations,  $Ax = b$ , using the Gauss elimination method. The function should return an error message if the determinant of  $A$  vanishes.
- (c) Redo the above task using the Gauss-Jordan method.
- (d) Write a function to calculate the inverse of a matrix using the Gauss-Jordan method.

Note: Your codes should work for linear systems of arbitrary size.

---

## 5 Power Method for Eigenvalues of Real Symmetric Matrices

- (a) Given a real symmetric  $n \times n$  matrix,  $A$  from the standard input find its maximum eigenvalue (in absolute value) and the corresponding eigenvector.
- (b) Repeat the above part by taking the input from a file and writing the output to another file.

HINTS: Start with any guessed eigenvector,  $\vec{x}$ . Divide  $\vec{x}$  by its largest magnitude component. Obtain the vector  $\vec{y} = A\vec{x}$ . Calculate  $m_0 = \vec{x} \cdot \vec{x}$ ,  $m_1 = \vec{x} \cdot \vec{y}$ , and,  $m_2 = \vec{y} \cdot \vec{y}$ . The guessed eigenvalue is  $m_1/m_0$  and the maximum error in the guessed eigenvalue is  $\sqrt{\left| \frac{m_2}{m_0} - q^2 \right|}$ . Replace  $\vec{x}$  with  $\vec{y}$  and repeat until the error becomes small. The guessed eigenvector is  $\vec{y}$ .

# Part II

## Theory

### References:

1. Press et. al., *Numerical Recipes in Fortran*
2. Kreyszig, *Advanced Engineering Mathematics*

## 1 Two-dimensional Newton-Raphson Method

Problem: Find a root of  $\vec{F}(\vec{x}) = 0$  using a guess value  $\vec{x}_0$ .

Expanding  $\vec{F}(\vec{x})$  in a Taylor series to linear order, we have,

$$\vec{F}(\vec{x} + \delta\vec{x}) = \vec{F}(\vec{x}) + J(\vec{x})\delta\vec{x}, \quad (1)$$

where  $J(\vec{x})$  is the Jacobian matrix, i.e.,  $J_{ij} = \frac{\partial F_i}{\partial x_j}$ . If  $\vec{x} + \delta\vec{x}$  is a root, then the left hand side of the above equation must vanish. Then, we have,

$$J\delta\vec{x} = -\vec{F} \Rightarrow \delta\vec{x} = -J^{-1}\vec{F}. \quad (2)$$

Thus, the algorithm for finding the root is to change  $\vec{x}$  following this rule until convergence is achieved.

$$\delta\vec{x}_{n+1} = \vec{x}_n - J^{-1}\vec{F}. \quad (3)$$

For a two-dimensional problem,  $J$  is a  $2 \times 2$  matrix. Thus, if,

$$J(\vec{x}) = \begin{pmatrix} a(\vec{x}) & b(\vec{x}) \\ c(\vec{x}) & d(\vec{x}) \end{pmatrix}, \text{ then, } J^{-1}(\vec{x}) = \frac{1}{ad-bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}. \quad (4)$$

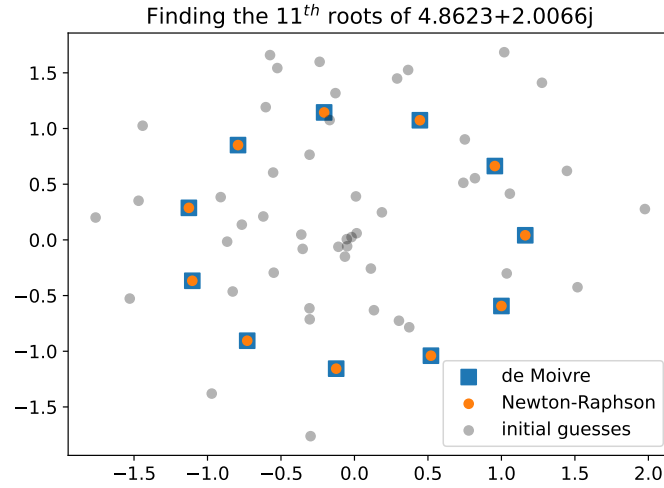


Figure 1: Sample calculation of roots with two-dimesional Newton-Raphson method and comparison with exact answers.

## 2 Improper Integrals

Consider the integrals,  $I_c(R) = \oint_C f(z)dz$  and  $I_r(R) = \int_{-R}^R f(x)dx$ ,  $C$  being a closed contour going from  $-R$  to

$R$  along the real axis and coming back along a semi-circular contour of radius  $R$  centered at the origin and on the upper half of the complex plane. In many situations, the contribution to the semicircular part,  $I_{sc}(R)$ , vanishes as  $R \rightarrow \infty$  and the value of the integral over the combined (closed) contour becomes equal to the

contribution from the real axis, i.e.,  $I \equiv \int_{-\infty}^{\infty} f(x)dx = \lim_{R \rightarrow \infty} I_r(R)$ . In Fig. 2, for the function  $f(x) = \frac{1}{1+x^2}$ , the variation of the real and imaginary parts of the above integrals are plotted as a function of  $R$ .

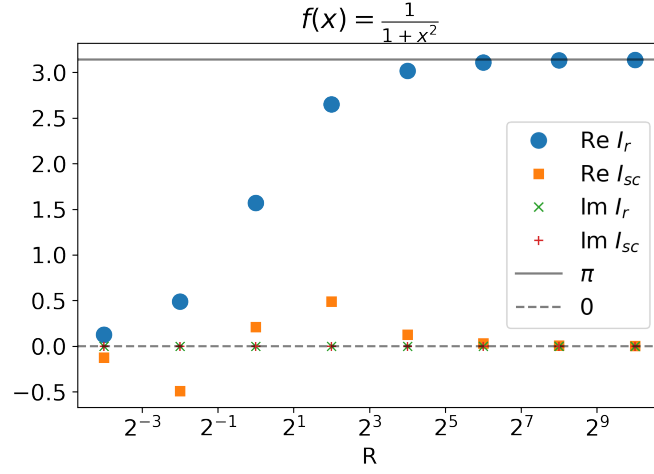


Figure 2: Variation of the real and imaginary parts of  $I_r$  and  $I_{sc}$  with  $R$ .

### 3 Gram-Schmidt Orthogonalization

Goal: Given a set of vectors,  $\{\vec{x}_i, i = 0, 1, 2, \dots, n-1\}$ , we need to find a set of orthogonal vectors,  $\{\vec{x}_{io}, i = 0, 1, 2, \dots, n-1\}$ , by forming suitable linear combinations of the vectors in the original set.

#### Algorithm

1.  $\vec{x}_{0o} = \vec{x}_0$
2. For  $i \geq 1$ ,  $\vec{x}_{io} = \vec{x}_i - \sum_{j=0}^{i-1} \vec{x}_i \cdot \hat{x}_{jo} \hat{x}_{jo}$ , i.e., we remove the components along the previously found vectors in the orthogonal set.

### 4 Gauss Elimination and Gauss-Jordan methods

Goal: Given an  $n \times n$  matrix,  $\mathbf{A}$ , and an  $n$ -component vector  $\mathbf{b}$ , calculate  $\det \mathbf{A}$  and solve the system  $\mathbf{Ax} = \mathbf{b}$ .

#### 4.1 Gauss Elimination

Method: use row operations to make an upper triangular matrix and then back-substitute (just like we did in school with pen and paper).

#### Algorithm

1. Horizontally stack  $\mathbf{A}$ , and  $\mathbf{b}$  and form the  $n \times (n+1)$  matrix  $\mathcal{A}$ .
2. *Conditioning*
  - (a) For each row of  $\mathbf{A}$ , find the elements which are largest in absolute value. This forms the vector  $\mathbf{a}$ .
  - (b) Divide each row of  $\mathcal{A}$  by the corresponding element of  $\mathbf{a}$ .
3. Let  $s = 1$ .
4. For  $i$  in  $0, 1, 2, \dots, n-2$ ,
  - (a) *Pivot*: Find  $j \geq i$ , such that for the  $i^{\text{th}}$  column, excluding the elements above the diagonal, the  $j^{\text{th}}$  row has the largest element in absolute value.

- (b) If the element which is largest in absolute value smaller than some threshold, e.g.,  $10^{-10}$ , then the matrix is singular. Stop with some error message.
  - (c) If  $j > i$ , swap the  $i^{\text{th}}$  and  $j^{\text{th}}$  rows and change the sign of  $s$ .
  - (d) For  $j$  in  $i + 1, i + 2, \dots, n - 1$ , subtract from the  $j^{\text{th}}$  row,  $\left(\frac{\mathcal{A}_{ij}}{\mathcal{A}_{ii}}\right)$  times the  $i^{\text{th}}$  row.
5. The value of the determinant is  $s$  times the product of the diagonal elements of  $\mathcal{A}$  times the product of the elements of the vector  $\mathbf{a}$ .
6. *Back-substitution:*
- (a)  $x_{n-1} = \frac{\mathcal{A}_{n-1,n}}{\mathcal{A}_{n-1,n-1}}$
  - (b) For  $i$  in  $n - 2, n - 1, \dots, 0$ ,  $x_i = \frac{\mathcal{A}_{i,n} - \sum_{j=i+1}^{n-1} \mathcal{A}_{i,j}x_j}{\mathcal{A}_{i,i}}$

## 4.2 Gauss-Jordan method

Method: use row operations to make an identity matrix.

### Algorithm

1. Horizontally stack  $\mathbf{A}$ ,  $\mathbf{b}$  and an  $n \times n$  identity matrix to form the  $n \times (2n + 1)$  matrix  $\mathcal{A}$ .
2. *Conditioning* Same as Gauss elimination.
3. Let  $s = 1$ .
4. For  $i$  in  $0, 1, 2, \dots, n - 2$ ,
  - (a) *Pivot:* Find  $j \geq i$ , such that for the  $i^{\text{th}}$  column, excluding the elements above the diagonal, the  $j^{\text{th}}$  row has the largest element in absolute value.
  - (b) If the element which is largest in absolute value smaller than some threshold, e.g.,  $10^{-10}$ , then the matrix is singular. Stop with some error message.
  - (c) If  $j > i$ , swap the  $i^{\text{th}}$  and  $j^{\text{th}}$  rows and change the sign of  $s$ .
  - (d) Multiply each element of  $\mathbf{a}$  with the corresponding diagonal element of  $\mathcal{A}$ .
  - (e) Divide the  $i^{\text{th}}$  row by  $\mathcal{A}_{ii}$ .
  - (f) For  $j$  in  $0, 1, 2, \dots, i - 1, i + 1, i + 2, \dots, n - 1$ , subtract from the  $j^{\text{th}}$  row,  $\mathcal{A}_{ij}$  times the  $i^{\text{th}}$  row.
5. The value of the determinant is  $s$  times the product of the elements of the vector  $\mathbf{a}$ .
6. The last column of  $\mathcal{A}$  gives the value of  $\mathbf{x}$ .
7. The previous  $n$  columns contain  $\mathbf{A}^{-1}$ .

## 5 Power Method for Eigenvalues

Suppose we want to find an eigenvalue and an eigenvector of some real symmetric matrix  $A$ . We will see that if we apply  $A$  to any non-zero vector  $x$  sufficient number of times, we get an eigenvector of  $A$ .

Suppose  $\vec{x}$  is some vector (our initial guess for the eigenvector).

Let  $\vec{y} = A\vec{x}$ .

Define  $m_0 = \vec{x} \cdot \vec{x}$ ,  $m_1 = \vec{x} \cdot \vec{y} \equiv qm_0$  and  $m_2 = \vec{y} \cdot \vec{y}$ .

Now, the  $n \times n$  real symmetric matrix  $A$  has  $n$  orthonormal eigenvectors.

Let  $\{\vec{z}_i\}$  denote the eigenvectors and  $\{\lambda_i\}$  the eigenvalues.

The vector  $\vec{x}$  may be expressed as a linear combination of the eigenvectors, i.e.,  $\vec{x} = \sum_i a_i \vec{z}_i$ .

$$\Rightarrow m_0 = \vec{x} \cdot \vec{x} = \sum_i a_i^2.$$

Also,  $\vec{y} = A\vec{x} = \sum_i a_i \lambda_i \vec{z}_i$ .

It is easy to see that  $(\vec{y} - q\vec{x}) \cdot (\vec{y} - q\vec{x}) = m_2 - q^2 m_0$ .

Now, suppose  $(\lambda_c - q)^2 = \min_i (\lambda_i - q)^2$ , i.e.,  $\lambda_c$  is the eigenvalue which is closest to  $q$ .

Then,  $(\vec{y} - q\vec{x}) \cdot (\vec{y} - q\vec{x}) = \sum_i a_i^2 (\lambda_i - q)^2 \geq \sum_i a_i^2 (\lambda_c - q)^2 = (\lambda_c - q)^2 m_0$ .

$\Rightarrow (\lambda_c - q)^2 \leq \frac{m_2}{m_0} - q^2$ .

Thus, after some iterations, if  $\frac{m_2}{m_0} - q^2$  becomes sufficiently small, then an eigenvalue of  $A$  becomes close to  $q$ .

Then,  $\vec{x}$  becomes the corresponding eigenvector.